

Relazione elaborato di Architettura degli Elaboratori - SIS -



UniVR - Dipartimento di Informatica
A.A. 2023/2024

Componenti del gruppo:

Maria Marino – VR500489

Jessica Ornella Ravindraraj - VR508128

Indice

1. Specifiche del progetto
2. Architettura generale del circuito (schema FSM/D);
3. Diagramma degli stati del controllore;
4. Architettura del datapath;
5. Statistiche del circuito prima e dopo l'ottimizzazione per area;
6. Numero di gate e ritardo ottenuti mappando il design sulla libreria tecnologica synch.genlib;
7. Descrizione di eventuali scelte progettuali effettuate

Specifiche del progetto

Si progetti un dispositivo per la gestione di partite della morra cinese, conosciuta anche come sasso-carta-forbici.

Due giocatori inseriscono una mossa, che può essere carta, sasso, o forbici. Ad ogni manche, il giocatore vincente è decretato dalle seguenti regole:

- Sasso batte forbici;
- Forbici batte carta;
- Carta batte sasso.

Nel caso in cui i due giocatori scelgano la stessa mossa, la manche finisce in pareggio.

Per renderla più avvincente, ogni partita si articola di più manche, con le seguenti regole:

- Si devono giocare un minimo di quattro manche;
- Si possono giocare un massimo di diciannove manche. Il numero massimo di manche viene settato al ciclo di clock in cui viene iniziata la partita;
- Vince il primo giocatore a riuscire a vincere due manche in più del proprio avversario, a patto di aver giocato almeno quattro partite;
- Ad ogni manche, il giocatore vincente della manche precedente non può ripetere l'ultima mossa utilizzata. Nel caso lo facesse, la manche non sarebbe valida ed andrebbe ripetuta (quindi, non conteggiata);
- Ad ogni manche, in caso di pareggio la manche viene conteggiata. Alla manche successiva, entrambi i giocatori possono usare tutte le mosse.

Il circuito ha tre ingressi:

- PRIMO [2 bit]: mossa scelta dal primo giocatore. Le mosse hanno i seguenti codici:
 - 00: nessuna mossa;
 - 01: Sasso;
 - 10: Carta;
 - 11: Forbice;
- SECONDO [2bit]: mossa scelta dal secondo giocatore. Le mosse hanno gli stessi codici del primo giocatore.
- INIZIA [1 bit]: quando vale 1, riporta il sistema alla configurazione iniziale. Inoltre, la concatenazione degli ingressi PRIMO e SECONDO viene usata per specificare il numero massimo di partite oltre le quattro obbligatorie. Ad esempio, se si inserissero i valori PRIMO = 00 e SECONDO = 00, si indicherebbe di giocare esattamente quattro partite. Se si inserisse il valore PRIMO = 00 e SECONDO = 01, si indicherebbe di giocare al più 5 partite (le 4 obbligatorie, più il valore 1 indicato da 0001). Se si inserissero i valori PRIMO = 10 e SECONDO = 01, si indicherebbe di giocare tredici partite (le 4 obbligatorie, più il valore 9 indicato da 1001). Quando vale 0, la partita prosegue normalmente.

Il circuito ha due uscite:

- MANCHE [2 bit]: fornisce il risultato dell'ultima manche giocata con la seguente codifica:
 - 00: manche non valida;
 - 01: manche vinta dal giocatore 1;
 - 10: manche vinta dal giocatore 2;

- 11: manche pareggiata.
- PARTITA [2 bit]: fornisce il risultato della partita con la seguente codifica:
 - 00: la partita non è terminata;
 - 01: la partita è terminata, ed ha vinto il giocatore 1;
 - 10: la partita è terminata, ed ha vinto il giocatore 2;
 - 11: la partita è terminata in pareggio.

Il circuito deve essere implementato in Verilog (nello stile behavioral) ed in SIS. Gli ingressi e uscite delle implementazioni Verilog e SIS devono avere lo stesso ordine riportato sopra. Il modulo principale Verilog dovrà chiamarsi Morra Cinese. Il testbench Verilog deve generare, mediante stampa su file chiamato testbench.script, uno script che funga da testbench per il modello SIS.

Il modello Verilog deve generare un file, chiamato output_verilog.txt che ad ogni ciclo di clock riporta gli output con il seguente formato:

Outputs: MANCHE[1] MANCHE[0] PARTITA[1] PARTITA[0]

Architettura generale del circuito (schema FSMD)

Il circuito è composto da una FSM (controllore) e un DATAPATH (elaboratore) che comunicano tra loro.

I file che contengono la rappresentazione di queste componenti sono presenti nella cartella denominati FSM.blif e DATAPATH.blif.

Il file che permette il collegamento tra la macchina a stati finiti e l'elaboratore ha il nome di FSMD.blif. Nelle pagine seguenti verranno descritte nel dettaglio le componenti utilizzate.

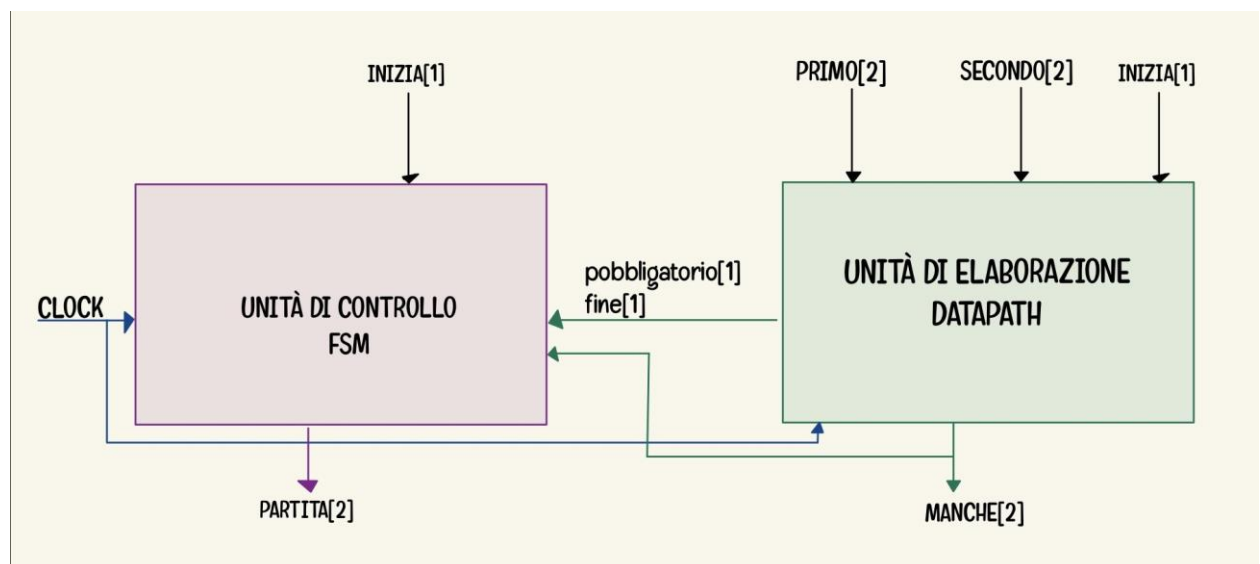


figura 1: schema FSMD

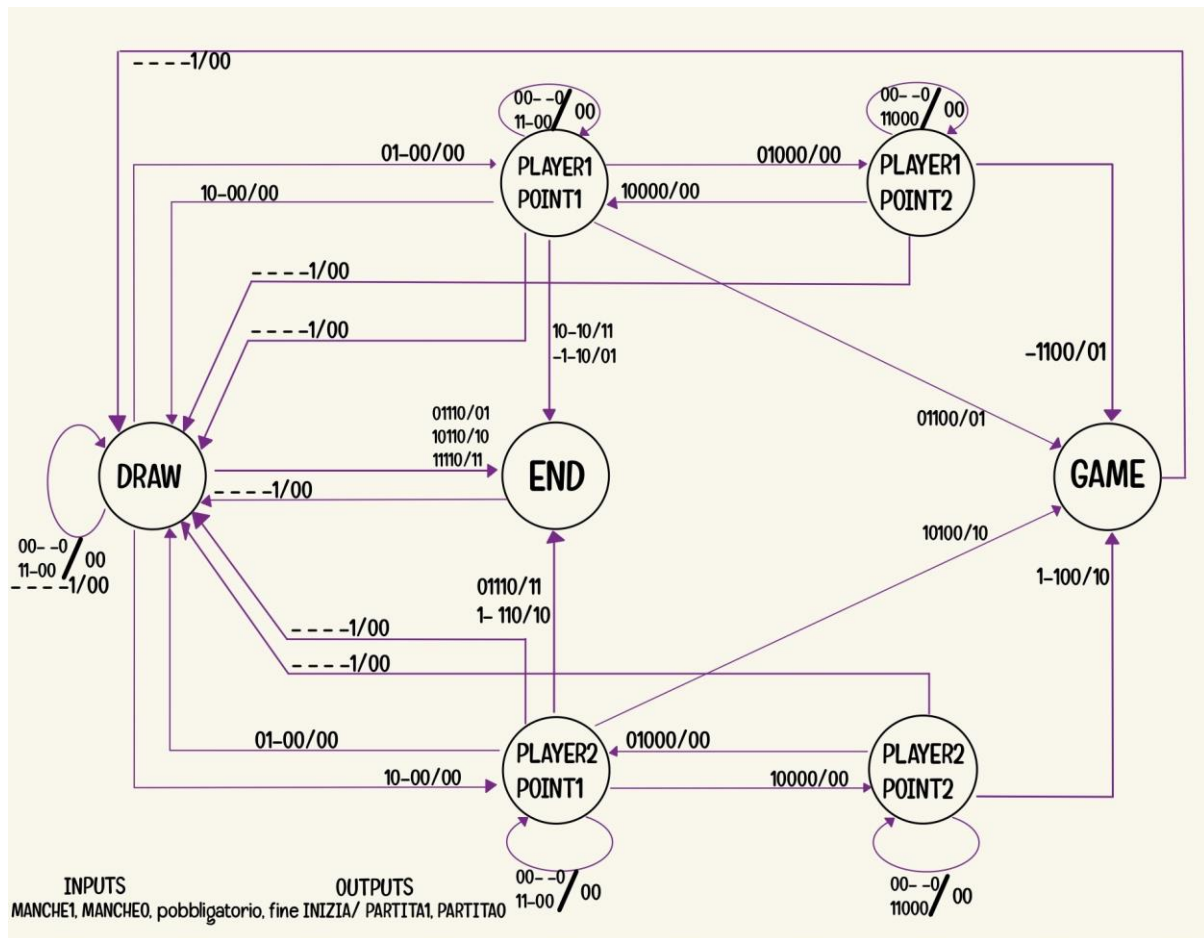
Diagramma degli stati del controllore

Il controllore della Morra Cinese è una macchina a stati finiti di tipo Mealy.
Si presentano i seguenti segnali:

SEGNALI DI INPUT	SEGNALI DI OUTPUT
MANCHE[2]	PARTITA[2]
pobbligatorio[1]	
fine[1]	
INIZIA[1]	

La FSM presenta 7 stati:

- **DRAW:**
 - È lo stato iniziale
 - Se si trova in END oppure GAME (indica che la partita è terminata), all'inizializzazione di una nuova partita si ritorna in questo stato
- **PLAYER1POIN1:**
 - Indica che il primo giocatore è in vantaggio di 1 punto rispetto all'avversario
- **PLAYER1POIN2:**
 - Indica che il primo giocatore è in vantaggio di 2 punti rispetto all'avversario
- **PLAYER2POIN1:**
 - Indica che il secondo giocatore è in vantaggio di 1 punto rispetto all'avversario.
- **PLAYER2POIN2:**
 - Indica che il secondo giocatore è in vantaggio di 1 punto rispetto all'avversario.
- **GAME:**
 - Si arriva in questo stato se uno dei giocati ha un vantaggio di 2 punti e le quattro manche obbligatorie sono state giocate, indicando quindi il termine della partita
- **END:**
 - Si arriva in questo stato se uno dei giocati ha un vantaggio di 1 punto e la partita fosse finita
 - Si arriva qua anche in caso di punteggio pareggiato
- Il segnale fine indica il termine della partita, in quanto tutte le manche sono state giocate
- Il segnale pobbligatorio indica che le quattro manche sono state giocate
- Il segnale INIZIA resetta tutto



- **SELECT**: In base alla validità della manche il mux aggiorna il contatore.
- **RESETCONTATORE**: In base al segnale di selezione INIZIA il mux viene resettato oppure prende il valore corrente arrivato dal sommatore.
- **MOSSA1**: In base al segnale attribuisce il valore (mossa del 1'giocatore) al registro MOSSA1.
- **MOSSA2**: In base al segnale attribuisce il valore (mossa del 2'giocatore) al registro MOSSA2.
- **VICTORYMANCHE**: decreta il risultato della manche avvenuta.
- 2 SOMMATORI:
 - Per decidere il totale delle manche da giocare in aggiunta con le quattro manche obbligatorie
 - Contare ogni manche valida giocata
- 4 COMPARATORI:
 - Due comparatori servono per definire il termine della partita e per controllare se le effettive manche obbligatorie sono state giocate.
 - I restanti cinque servono per decretare chi ha più manche vinte.
- COMPONENTI LOGICI: servono per un parziale decreto del vincitore della manche.

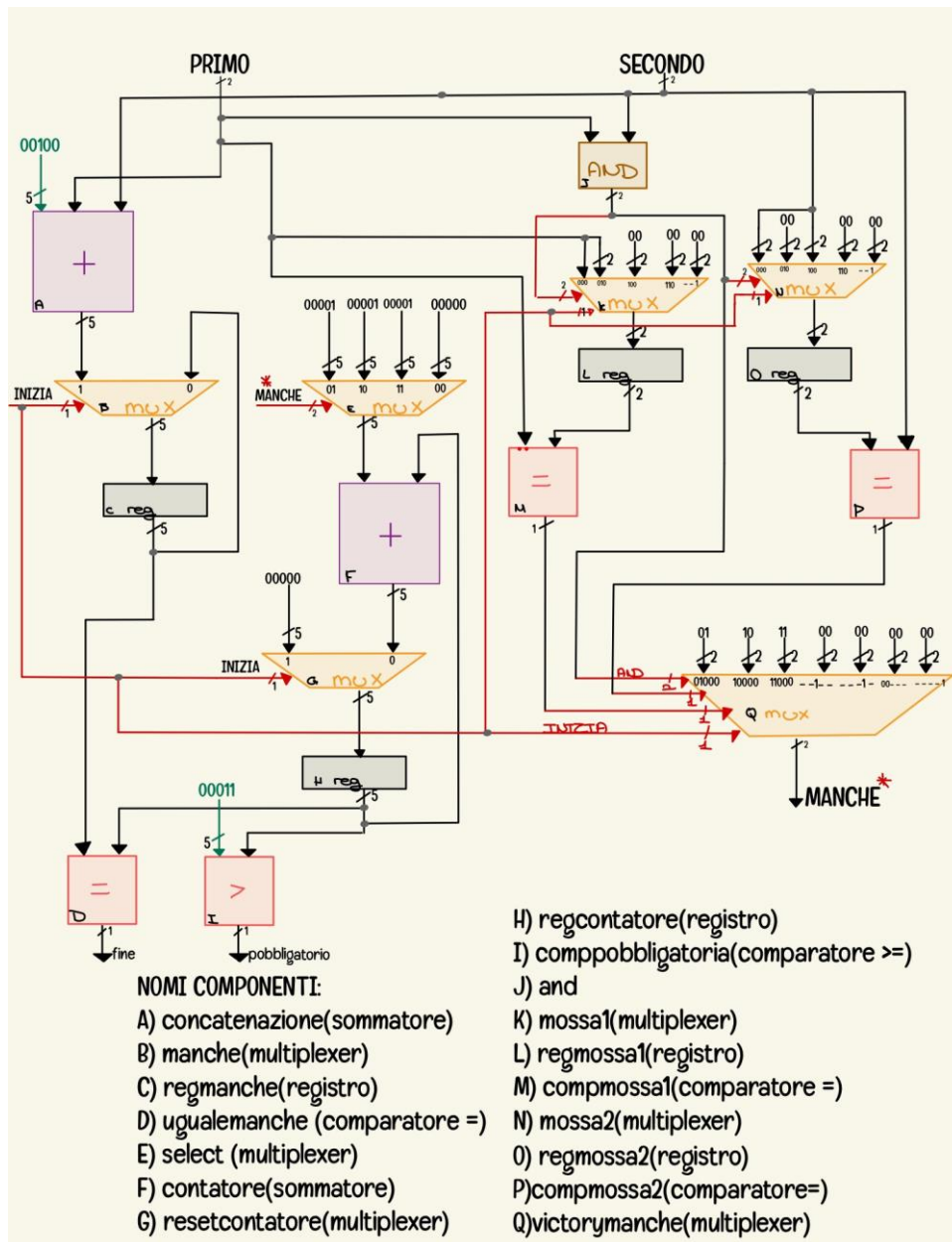


figure 3: schema datapath

Statistiche del circuito prima e dopo l'ottimizzazione per area

```
sis> print_stats
morracinese      pi= 5   po= 4   nodes= 80      latches=17
lits(sop)= 656
sis> full_simplify
sis> print_stats
morracinese      pi= 5   po= 4   nodes= 80      latches=17
lits(sop)= 282
sis> source script.rugged
sis> source script.rugged
sis> print_stats
morracinese      pi= 5   po= 4   nodes= 39      latches=17
lits(sop)= 286
sis> source script.rugged
sis> print_stats
morracinese      pi= 5   po= 4   nodes= 39      latches=17
lits(sop)= 286
sis> write_blif FSMD.blif
```

Lo script.rugged era stato eseguito tre volte.

Numero di gate e ritardo ottenuti mappando il design sulla libreria tecnologica synch.genlib

Una volta ottimizzato il circuito, lo abbiamo mappato con una libreria, in modo da avere delle statistiche più verosimili per quanto riguarda area e ritardo. Nel nostro caso la libreria assegnata è la “synch.genlib”. Una volta mappato il circuito presenta le seguenti statistiche:

```
sis> read_library synch.genlib
sis> map -m 0
warning: unknown latch type at node '{[12]}' (RISING_EDGE assumed)
warning: unknown latch type at node '{[13]}' (RISING_EDGE assumed)
warning: unknown latch type at node '{[14]}' (RISING_EDGE assumed)
WARNING: uses as primary input drive the value (0.20,0.20)
WARNING: uses as primary input arrival the value (0.00,0.00)
WARNING: uses as primary input max load limit the value (999.00)
WARNING: uses as primary output required the value (0.00,0.00)
WARNING: uses as primary output load the value 1.00
```

```

sis> map -s
>>> before removing serial inverters <<<
# of outputs:      21
total gate area:    5144.00
maximum arrival time: (47.60,47.60)
maximum po slack:   (-5.40,-5.40)
minimum po slack:   (-47.60,-47.60)
total neg slack:    (-412.20,-412.20)
# of failing outputs: 21
>>> before removing parallel inverters <<<
# of outputs:      21
total gate area:    5128.00
maximum arrival time: (47.60,47.60)
maximum po slack:   (-5.40,-5.40)
minimum po slack:   (-47.60,-47.60)
total neg slack:    (-412.40,-412.40)
# of failing outputs: 21
# of outputs:      21
total gate area:    4904.00
maximum arrival time: (46.40,46.40)
maximum po slack:   (-5.40,-5.40)
minimum po slack:   (-46.40,-46.40)
total neg slack:    (-403.20,-403.20)
# of failing outputs: 21
sis> read_blif FSMD.blif
sis> quit

```

Scelte progettuali effettuate

Implementando il nostro progetto abbiamo riscontrato alcuni punti che risultavano essere non del tutto chiari, non essendo presenti alcune indicazioni nelle specifiche ci siamo permesse di affermare alcune ipotesi per quanto riguarda il funzionamento del nostro circuito che ora elenchiamo:

1. Chi vince più manche vince la partita:
Qualora ci si trovasse in una situazione in cui i due giocatori avessero giocato le quattro partite obbligatorie, e si trovassero in pareggio con un'ultima manche da giocare, vince il primo giocatore che vince la manche. Ad esempio, se i giocatori scegliessero di giocare sette manche e arrivati alla settima manche il loro punteggio dovesse essere 3- 3(pertanto non è presente la differenza di due punti in più dell'avversario), chi vince la settima manche vince la partita.
2. La codifica 00(nessuna mossa) rappresenta:
 - a. In caso di inizializzazione della partita, i due giocatori che intendono solamente giocare le quattro manche obbligatorie.

- b. In caso di vincita di uno dei due giocati, il blocco mossa per il riutilizzo.
- 3. Al raggiungimento della differenza di due punti la partita termina:
Se prima della fine della partita (cioè giocare tutte le manche stabilite) un giocatore fosse in vantaggio di due punti rispetto all'avversario, egli viene decretato vincitore e la partita termina (anche se non si sono giocate tutte le manche).